

注意力逐项计算实验

第052单元 独立操作与验收

本实验要求把纸面矩阵、独立NumPy、PyTorch自动微分、两种注意力API和信息边界连接起来。先读lecture前八节，再完成A至D；阅读mask章节后完成E至H。先写自己的预测与计算，再查看answers。仅仅运行Notebook或看到15个测试通过，不等于已经理解。

需要012的Jacobian/VJP、019的概率支持集、049的padding和目标位移。实验数据均为原创合成值，无需下载外部数据。课程不调用网络训练服务，无API密钥。普通实验在CPU上运行；具体作者环境和含写盘耗时见outputs/runtime.json，不承诺不同机器耗时相同。

一 安装与首次运行

从课程根目录执行。已有独立Python环境也可核对environment.yml的版本后使用；不要混用不同环境的python和pip。

```
conda env create -f units/052/environment.yml
conda activate dl052
python -c "import torch,numpy; print(torch.__version__,numpy.__version__)"
export OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1
mkdir -p tmp/052-replay/mpl tmp/052-replay/cache
export MPLCONFIGDIR="$PWD/tmp/052-replay/mpl"
export XDG_CACHE_HOME="$PWD/tmp/052-replay/cache"
```

完整Notebook和make_figures.py需要Noto Sans CJK系统字体。environment.yml只列Python依赖，不安装字体。代码默认读取/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc，并在缺失时明确报错；不会默默生成缺字图。Debian/Ubuntu可从官方系统包安装：

```
sudo apt-get update
sudo apt-get install fonts-noto-cjk
python -c "from pathlib import Path; p=Path('/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc'); print(p, p.is_file())"
```

其他系统可从Google Noto官方发行安装Noto Sans CJK，并设置DL_CJK_FONT为实际存在的ttc或otf文件路径。路径中有空格时加引号。随后检查代码实际使用的文件：

```
export DL_CJK_FONT="/your/actual/path/NotoSansCJK-Regular.ttc"
python -c "import os; from pathlib import Path; from matplotlib.font_manager import FontProperties; p=Path(os.environ['DL_CJK_FONT']); print(p.is_file(), FontProperties(fname=str(p)).get_name())"
```

上面/your/actual/path是需替换的示意，不是现成目录。Linux作者环境已实际运行，macOS/Windows完整安装及浏览器界面未测试。数值核心和test_experiment.py不导入绘图模块，因此无需字体即可检查数值；完整Notebook会导入绘图模块，需要字体。

```
python units/052/experiment.py --output tmp/052-replay
python units/052/test_experiment.py
python -O units/052/test_experiment.py
python units/052/make_figures.py --output tmp/052-replay/figures
jupyter lab units/052/experiment.ipynb
```

Notebook打开后Restart Kernel并Run All。可从课程根或units/052目录启动；其他工作目录会明确报错。发布Notebook已保存真实输出。create_notebook.py只用于重建未执行源，会清除旧执行结果，不应为了阅读先运行它。离线复验入口如下，使用新Python进程中的真实IPython核，不验证浏览器与socket：

```
python shared/execute_notebook_inprocess.py units/052/experiment.ipynb
```

不要删掉随包outputs再跑测试，因为测试会检查保留结果。重跑输出另放tmp目录，不覆盖原始证据。generate_data.py用于有意重建相同种子数据，一般学习不需要先运行。

二 固定实验协议与应交内容

先打开data/protocol.json，写下：128训练序列、512测试序列、长度4、种子5201至5203、 $\alpha=0$ 或8、因果或无mask、训练集最小二乘、2个训练参数、测试MSE、零基线。没有测试选模型或种子替换。保存所有12行结果，包括失败和重复结果。

交付内容应有：手写或电子矩阵、每个量的shape、两次同步更新账本、有限差分误差、三种mask对照、未来注入前后输出、12配置结果及一段限制说明。代码截图不能代替数值文件；只有总loss的截图也不能证明梯度正确。

A 加权和与一般shape

1. 候选 $V=(2,4,8)$ 和权重 $(0.2,0.3,0.5)$ 的输出是多少？把候选改成 $(2,-1)$ 、 $(4,3)$ 、 $(8,5)$ ，用同样权重求二维输出。
2. $B=2$ ， $L=3$ ， $S=4$ ， $dk=2$ ， $dv=3$ ，写出Q、K、V、S、A、O，以及GO、GA、GS、GQ、GK、GV的shape。哪几个轴负责归一化与梯度累加？
3. 用独立循环实现scalar_attention，不能调用attention或矩阵乘法。对同一随机QKV与mask，和矢量版本对齐。

验收：输出逐元素一致到 $1e-13$ 量级；有效行A和为1，禁止位置A严格为0。不把不同样本混起来归一化。

B 同一实例的前向与完整softmax导数

使用lecture第四节的两样本X、Y和三个W，禁止另换一个更简单样例。分别算Q、K、V、分数、A、O和四个误差。写清half-MSE分母8，以及GO为什么除以4。

对样本1位置1，从商法则求完整 2×2 Jacobian，把对角和非对角四项全部写出。用GAJ和逐元素VJP各算一次GS。再对 $a=(0.2,0.3,0.5)$ 写 3×3 Jacobian，检查每列和为0。说明只保留对角为何错误。

验收：初始总loss约0.016128563004；小数舍入误差与程序浮点误差分开报告。不能看到目标是0.5就把该样本的所有梯度都写0，它还有第二个位置。

C 每条共享路径与两次更新

从GO开始求GA、GV、GS、GQ、GK。选样本1的value1，列出两个query对它的贡献。选样本2的WK第1坐标，列出两个位置的参数贡献并解释抵消。

把每个位置的外积贡献累加到样本，再跨样本相加；三组W一起更新，学习率0.1。重算全部前向和全部反向，再做第二次更新，最后再做完整前向。记录三轮所有参数、A、O、loss，不复用旧A。

同时求GX的Q、K、V三路贡献，解释如果X来自重复词元嵌入时如何继续累加。不要更新X，因为这里它是数据。

验收：三轮loss依次约0.016128563004、0.015241267063、0.014489698297。指出至少一个误差变差的位置，解释共享参数折中。不能只报告总loss下降。

D 独立数值差分与自动微分

中心差分用 $h=1e-6$ ，对六个W坐标、八个X坐标逐个做两次前向。每次只扰动一个坐标，其余不变，用同一loss归约。分别与手写梯度和PyTorch autograd对齐。

写出每坐标解析导数、差分值和绝对误差。初始状态的X和W分别设requires_grad=True；PyTorch使用CPU float64，所有tensor常数声明dtype/device。清除旧梯度或新建叶子张量，不能累加上一次backward残留。

验收：本环境最大有限差分绝对误差约 $3.23e-12$ ，允许读者报告合理不同舍入误差；本课测试阈值 $1e-9$ 。过大误差先查平均因子、转置、共享路径、dtype和更新顺序，不直接放宽阈值。

E 三种mask与全屏蔽行

1. 长度4，只前三个key有效，写 4×4 因果允许矩阵和key有效矩阵，再取交集。
2. 把第4个query标无效，分别观察仅key mask与显式query约定。说明loss mask为什么仍需要。
3. 将一个有效query全部屏蔽，应得到ValueError。把它明确标无效，输出和梯度应为0而非NaN。说明此时A行和为什么是0。

4. 直接调用原始softmax、SDPA、MHA `need_weights=True/False`，记录全屏蔽行为。保留NaN诊断，不用`nan_to_num`。

验收：有效行空支持必须拒绝。数学未定义、实现约定和当前库观察要分开陈述；不能用某API返回零证明空集合softmax在数学上合法。

F API布尔语义对齐

使用`api_probes`的`dk=dv=2`、`L=2`、`S=3`例子，显式`allow`两行为`(True,False,True)`与`(False,True,True)`。先NumPy和标量手算，再调用SDPA与单头identity投影MHA。明确写出传给两个API的bool矩阵。

故意把SDPA收到的mask取反，观察数值差异。另传MHA `key_padding_mask`屏蔽最后一个key，检查第二个query仍非零。列出torch版本、设备、dtype、dropout和`need_weights`，不能只写“PyTorch就是这样”。

验收：正确转换时误差 $<1e-13$ ；错误转换产生显著差异。不要用随机MHA权重比较SDPA，因为MHA有额外投影。

G 未来注入与完整诊断实验

对`leakage_probe`的四位置输入，只更改位置2、3，打印前两个位置的两个输出坐标。对`causal`与`unmasked`分别计算前缀最大绝对差。若加入随机dropout，先控制随机性再比较；本任务固定无dropout。

完整运行12配置。独立用保存的`attention`、`coefficient`和测试原始值重算每个预测及MSE，不调用`leak_experiment`。为什么 $\alpha=0$ 无mask也有泄漏？为什么 $\alpha=0$ 与8的因果结果完全相同？为什么合法模型没胜过零基线不应被删掉？

验收：合法前缀变化0，无mask约7.616883398。合法模型三种子都没胜过零基线；无mask的低误差不能作为部署预测能力。实验只做描述性比较，不把三种子称作充分显著性证据。

H 注意力权重与研究结论

固定 $V=(0,1,2)$ ，比较 $A=(0.4,0.2,0.4)$ 与 $B=(0.2,0.6,0.2)$ 。两组权重能否产生同一输出？是否都能由softmax得到？这个反例能证明什么，不能证明什么？

提出一个有界研究扩展：例如固定现有拆分，改为未来与过去相关的AR(1)生成机制，预注册相关系数与合法基线，再重新训练。只写方案，不改本次已冻结协议；需要更多研究时另存新协议和新结果。

三 常见失败的定位顺序

先查shape和dtype，再查mask含义和信息边界，再查loss分母，最后查导数与更新。程序允许范围是非空rank3、每轴 ≤ 512 、float64有限值且绝对值 ≤ 10000 ，mask精确bool形状；极端域不支持就明确拒绝。异常消失不是正确性证据，尤其不能用全局NaN清洗来让测试变绿。

最终回答需明确：本地CPU已复现的内容、数学推导、当前API行为以及未测平台边界。作者执行检查不替代独立教学与科学审核。

四 最小可粘贴的API检查

完成手算后，可把下面代码放进课程根目录启动的Python或Notebook。它使用不相等的允许分数，因此既检查mask也检查缩放。全部浮点常数显式为CPU float64，bool mask单独声明类型。

```
import sys
sys.path.insert(0, 'units/052')
import numpy as np
import torch
import torch.nn.functional as F
from experiment import attention
kw = dict(dtype=torch.float64, device='cpu')
q = torch.tensor([[[1., 0.], [0., 1.]]], **kw)
k = torch.tensor([[[1., 0.], [0., 2.], [2., 1.]]], **kw)
v = torch.tensor([[[2., -1.], [4., 3.], [8., 5.]]], **kw)
allow = torch.tensor([[[True, False, True],
                        [False, True, True]],
                        dtype=torch.bool, device='cpu')
expected, cache = attention(q.numpy(), k.numpy(),
                             v.numpy(), allow.numpy())
actual = F.scaled_dot_product_attention(
    q, k, v, attn_mask=allow, dropout_p=0.0)
np.testing.assert_allclose(actual.numpy(), expected,
                             atol=1e-13, rtol=1e-13)
print(actual)
```

先预测去掉sqrt(2)、取反mask或屏蔽全部key分别会发生什么，再分别改一处验证。改实验副本，不覆盖原始results。MHA对照还需要identity投影和mask取反，完整实现在api_probes中，不能仅把函数名换成MHA便声称等价。